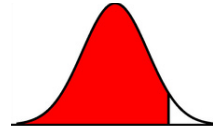
Next-Gen Credit Risk and Fraud Detection with Explainable AI

Cheng-Min Tsai,
Nguyen-Thu Thuy,
Huei-Wen Teng

NYCU
MY SITE

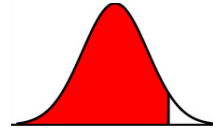
Outline

1. Motivation ✓
2. Overview Study Plan
3. Intro Datasets
4. Define Credit Fraud
5. Clean Data
6. EDA Chart
7. Conclusion

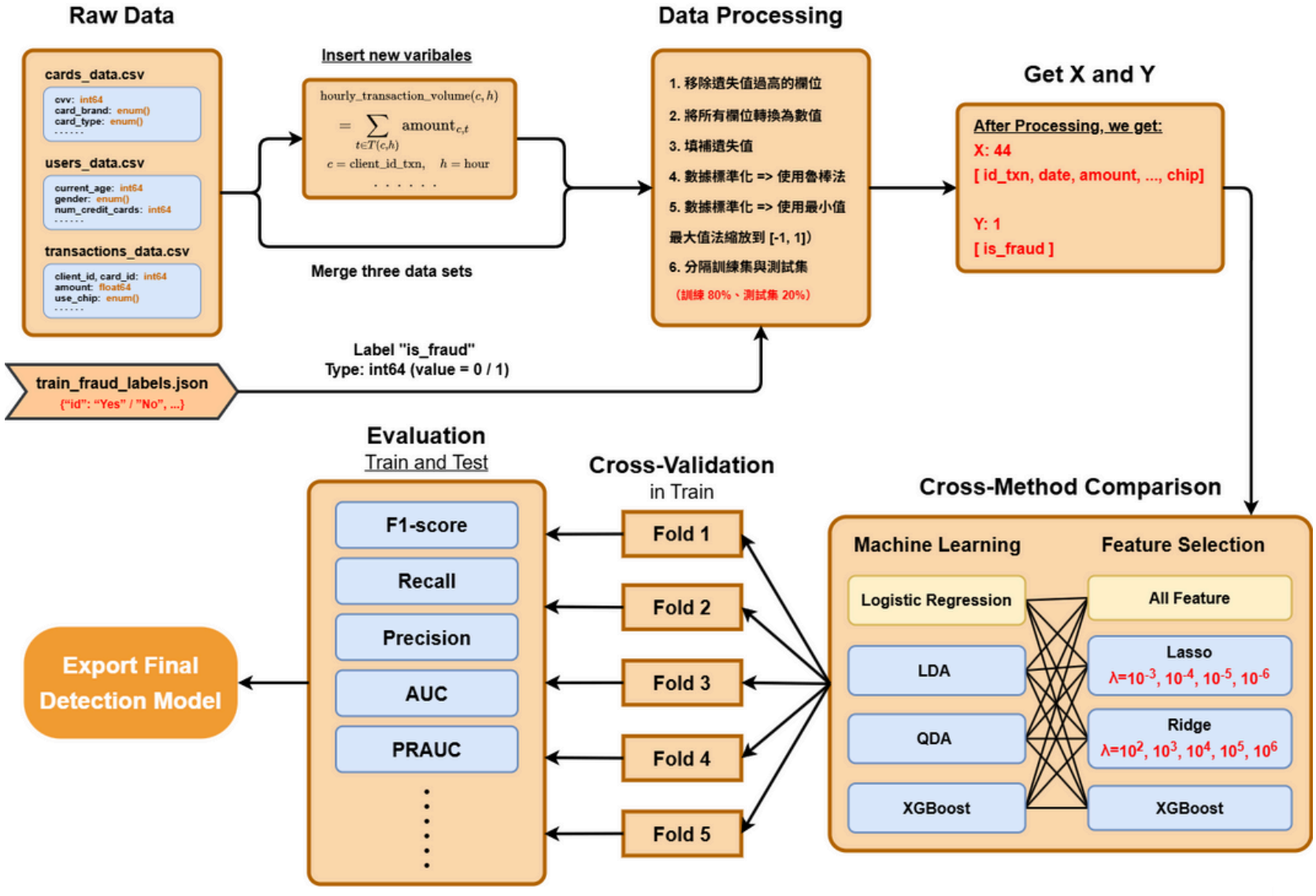


Motivation

- Digital finance creates high-volume, fast-changing credit and fraud risks.
- Traditional rule-based and linear models fail to capture complex behavioral patterns.
- Fraud schemes evolve quickly and are difficult to detect with static systems.
- Regulators require transparency, fairness, and explainable model decisions.
- Need models that are both accurate and interpretable for real-world deployment.
- This study explores machine learning with explainable AI to improve risk and fraud detection.



Overview Study Plan



Intro Datasets

- Teachers datasets on kaggle (Kaggle's Financial Transactions Dataset: Analytics, introduced by Chrissy)
- **Cards, Transactions, Users, train_fraud_labels.json**

	Item	Description
1	id	交易 ID
2	date	交易時間（YYYY/MM/DD HH:MM）
3	client_id	客戶 ID
4	card_id	卡片 ID
5	amount	交易金額
6	use_chip	交易方式（Chip Transaction / Swipe Transaction / Online）
7	merchant_id	商家 ID
8	merchant_city	商家所在城市
9	merchant_state	商家所在州
10	zip	商家郵遞區號
11	mcc	Merchant Category Code（商家類別碼）
12	errors	交易錯誤訊息

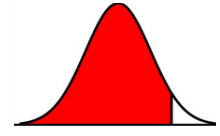
	項目	說明
1	id	卡片 ID
2	client_id	客戶 ID
3	card_brand	卡片品牌（如 Visa, MasterCard）
4	card_type	卡片類型（Debit / Credit）
5	card_number	卡號（數字格式）
6	expires	到期日（MM/YYYY）
7	cvv	安全碼
8	has_chip	是否有晶片
9	num_cards_issued	客戶已發行的卡片數量
10	credit_limit	信用額度（美元，需轉為數字）
11	acct_open_date	開卡日期（MM/YYYY）
12	year_pin_last_changed	最近一次修改 PIN 的年份
13	card_on_dark_web	卡片是否出現在暗網

	項目	說明
1	客戶 ID	客戶的唯一識別碼
2	目前年齡	客戶當前的年齡
3	預計退休年齡	客戶計劃退休的年齡
4	出生年份	客戶的出生年份
5	出生月份	客戶的出生月份
6	性別	客戶的性別
7	住址	客戶的居住地址
8	住家緯度	客戶住址的緯度座標
9	住家經度	客戶住址的經度座標
10	當地人均收入	客戶所在地區的平均人均收入
11	使用者年收入	客戶的年度總收入
12	總負債	客戶的總負債金額
13	信用評分	客戶的信用評分數值
14	擁有的信用卡數	客戶持有的信用卡數量

Clean the data

- There datasets we use is **Cards, Transactions, Users**
- Combine three datasets
- Remove unused datas (id_txn', 'card_id', 'merchant_id', 'date' ...)
- Remove columns with more than 70% missing values.
- Impute missing values with the median

	amount	merchant_city	merchant_state	zip	mcc	id_card	client_id_card	card_number
0	-77.00	0.000115	0.001944	58523.0	5499	2972	1556	5497590243197280
1	14.57	0.000025	0.013698	52722.0	5311	4575	561	5175842699412235
2	80.00	0.000476	0.121564	92084.0	4829	102	1129	5874992802287595
4	46.41	0.000203	0.016507	20776.0	5813	3915	848	4354185735186651
5	4.81	0.003918	0.073028	10464.0	5942	165	1807	5207231566469664
7	26.46	0.117540	0.033906	47710.0	4784	2140	1684	5955075527372953
10	3.51	0.000651	0.086072	77581.0	5311	3912	554	4096589319918041
11	2.58	0.006301	0.073028	11210.0	5411	5061	605	4799699589870743
12	39.63	0.000115	0.001944	58523.0	5499	2972	1556	5497590243197280
13	43.33	0.000061	0.006491	96732.0	4121	1127	1797	4777281869545650

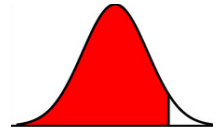


Clean the data

- Standardize the datas
  - Robust methods => Standardize the data using the median and interquartile range (IQR)
  - Min-Max scaling => Normalize the data to the range 0-1

```
print("\n==== Step 4: 數據標準化 ====")
X = df.drop(columns=[TARGET_COLUMN])
y = df[TARGET_COLUMN]

print("RobustScaler 執行中...")
X_robust = pd.DataFrame(RobustScaler().fit_transform(X), columns=X.columns)

print("MinMaxScaler 執行中...")
X_scaled = pd.DataFrame(MinMaxScaler().fit_transform(X_robust), columns=X.columns)
```

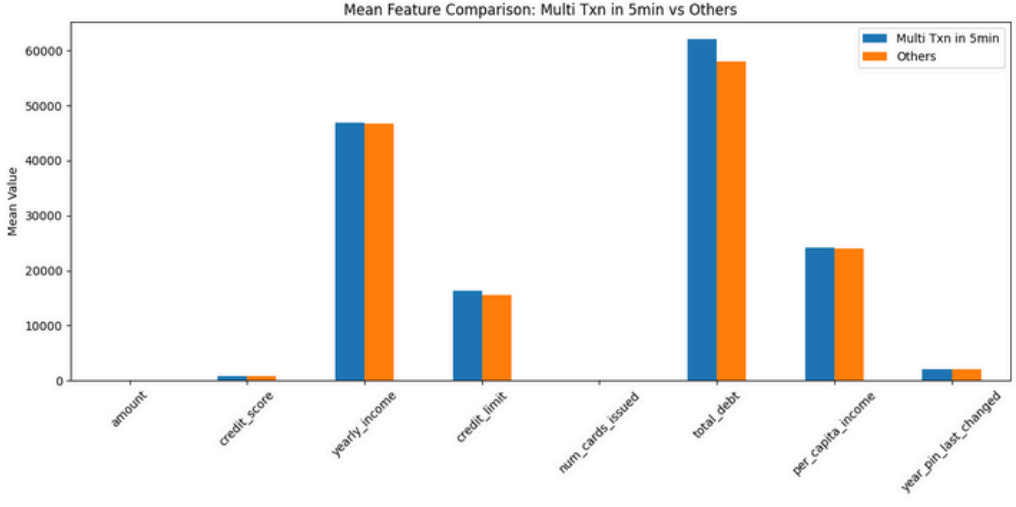
```
--- 最終訓練數據 (X_train, y_train) 檢查 ---
最終訓練集形狀: (6240474, 44)
目標欄位 is_fraud 的資料類型: int64
最終舞弊比例 (平均值): 0.0015

特徵 (X_train) 的統計概況 (前 5 欄):
       amount  merchant_city  merchant_state  zip  mcc
min  0.000000            0.0             0.0  0.0  0.0
max  0.871137            1.0             1.0  1.0  1.0
-----------------------------------
```

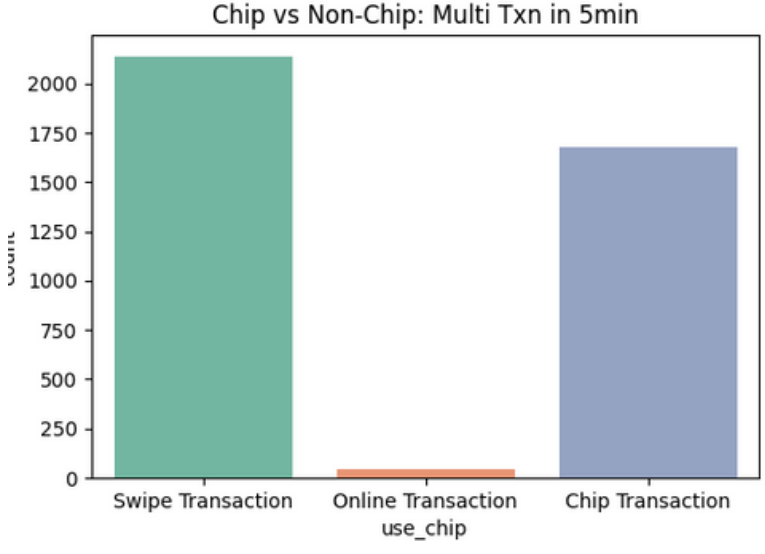


EDA Chart

- **Frequent transactions in a short period of time (5min)**
- Notated these datas in Trans_Data
- Measure the difference in these bias data

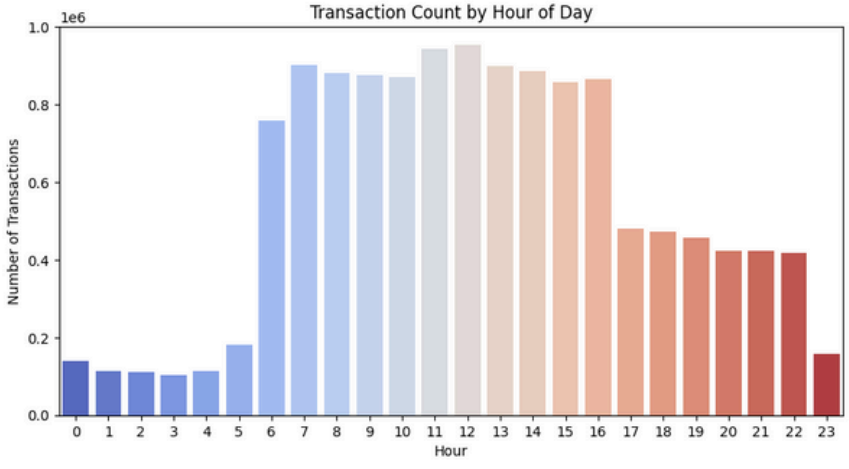
```
五分鐘內多筆交易（>3筆）共有 3862 筆，佔全部交易 0.03%
```

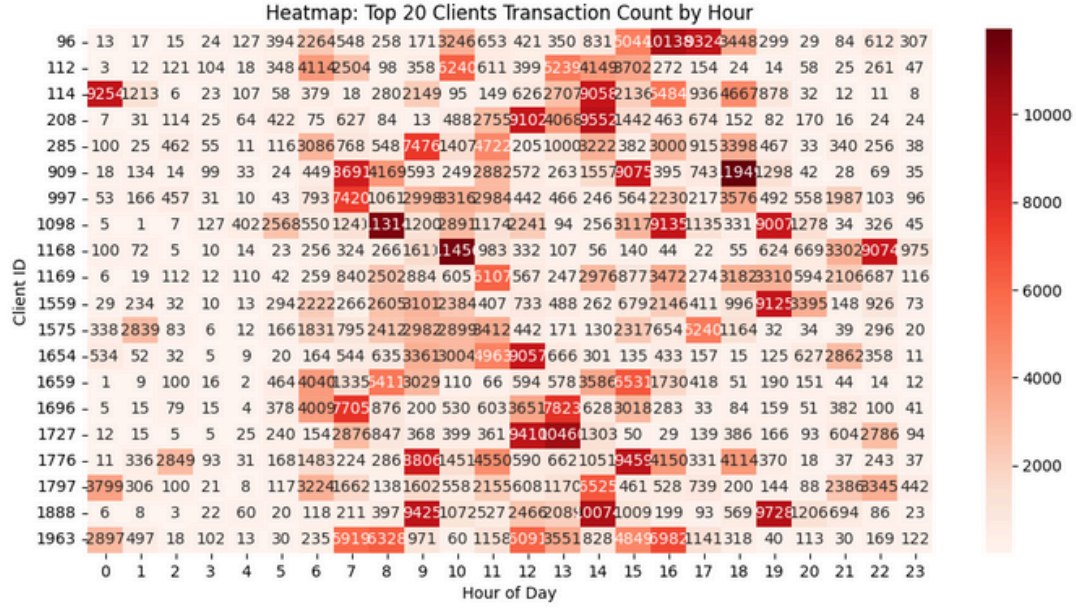




EDA Chart

- **About transactions Time**
- Observe numerous amount at strange time
  - Client 114
  - Client 1168





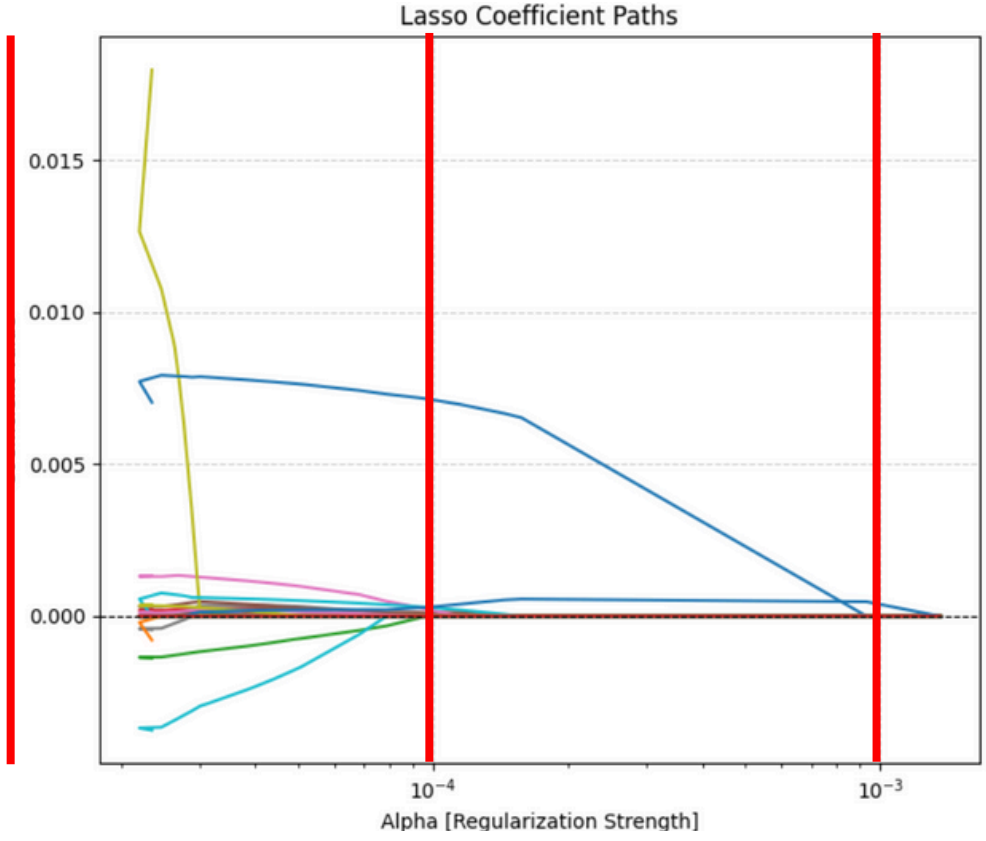
After EDA,

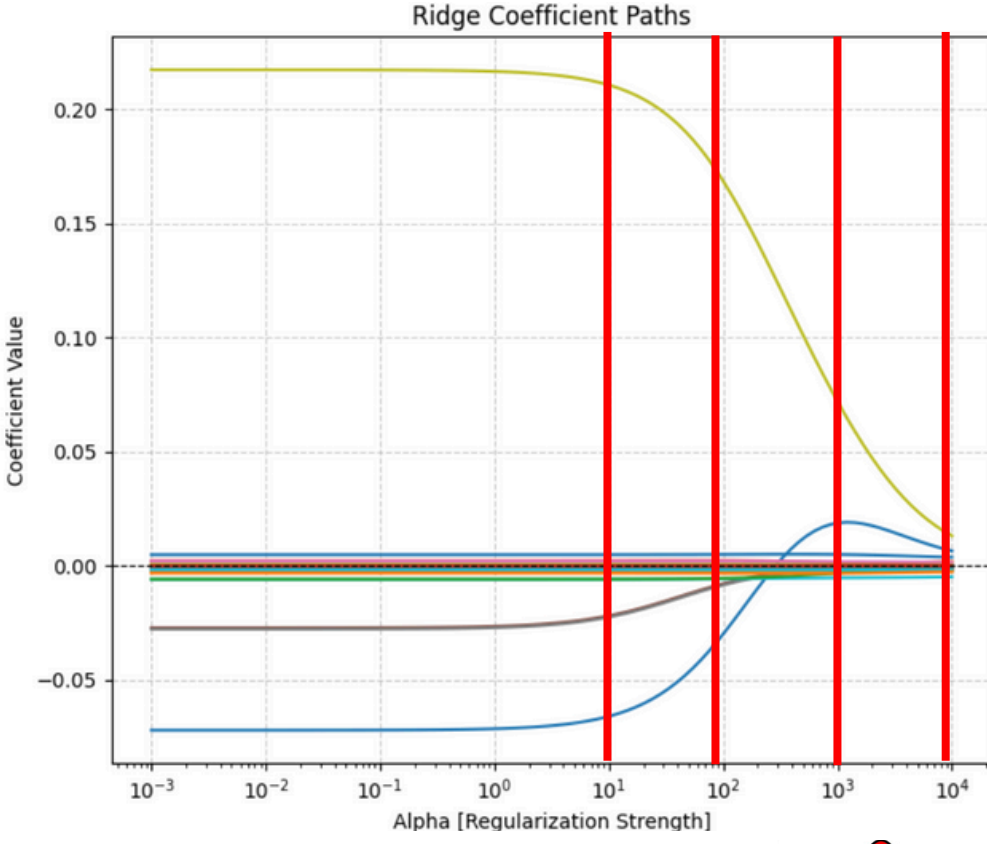
We found the transaction amount every hour is important indication.

Feature selection

Use All Features, Ridge, Lasso, and XGBoost

- **To find best alpha (λ) in Ridge and Lasso, we draw the chart**



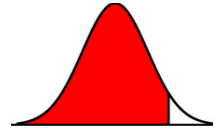


Model Training

choose four Machine Leaning Method

Finally, We cross 4 ML and 10 Feature sets => 40 combinations

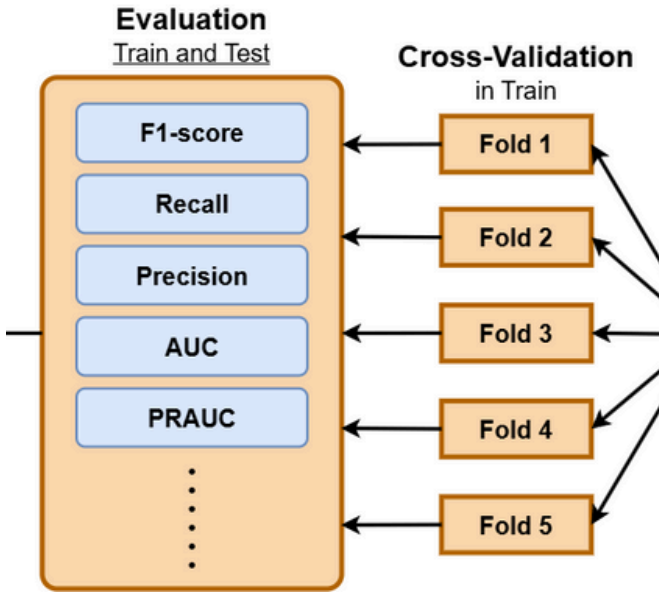
[1/40] Full_Set_Baseline + LogisticRegression
[2/40] Full_Set_Baseline + LDA
[3/40] Full_Set_Baseline + QDA
[4/40] Full_Set_Baseline + XGBoost
[5/40] Lasso (a=1e-6) + LogisticRegression
[6/40] Lasso (a=1e-6) + LDA
[7/40] Lasso (a=1e-6) + QDA
[8/40] Lasso (a=1e-6) + XGBoost
[9/40] Lasso (a=1e-5) + LogisticRegression
[10/40] Lasso (a=1e-5) + LDA
[11/40] Lasso (a=1e-5) + QDA
[12/40] Lasso (a=1e-5) + XGBoost
[13/40] Lasso (a=1e-4) + LogisticRegression
[14/40] Lasso (a=1e-4) + LDA
[15/40] Lasso (a=1e-4) + QDA
[16/40] Lasso (a=1e-4) + XGBoost
[17/40] Ridge (a=1e2) + LogisticRegression
[18/40] Ridge (a=1e2) + LDA
[19/40] Ridge (a=1e2) + QDA
[20/40] Ridge (a=1e2) + XGBoost
[21/40] Ridge (a=1e3) + LogisticRegression
[22/40] Ridge (a=1e3) + LDA
[23/40] Ridge (a=1e3) + QDA
[24/40] Ridge (a=1e3) + XGBoost
[25/40] Ridge (a=1e4) + LogisticRegression
[26/40] Ridge (a=1e4) + LDA
[27/40] Ridge (a=1e4) + QDA
[28/40] Ridge (a=1e4) + XGBoost
[29/40] Ridge (a=1e5) + LogisticRegression
[30/40] Ridge (a=1e5) + LDA
[31/40] Ridge (a=1e5) + QDA
[32/40] Ridge (a=1e5) + XGBoost
[33/40] Ridge (a=1e6) + LogisticRegression
[34/40] Ridge (a=1e6) + LDA
[35/40] Ridge (a=1e6) + QDA
[36/40] Ridge (a=1e6) + XGBoost
[37/40] XGBoost_Top10 + LogisticRegression
[38/40] XGBoost_Top10 + LDA
[39/40] XGBoost_Top10 + QDA
[40/40] XGBoost_Top10 + XGBoost



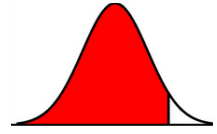
Model Training - Cross Validation

For training evaluation, we use 5-fold CV, and get the average.

=> The result is become more reliable

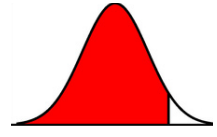



```
cv_results = cross_validate(
    model, X_train_fs, y_train,
    cv=cv_strategy,
    scoring=scoring_metrics,
    n_jobs=-1
)
train_accuracy  = cv_results['test_accuracy'].mean()
train_f1        = cv_results['test_f1'].mean()
train_precision = cv_results['test_precision'].mean()
train_recall    = cv_results['test_recall'].mean()
train_auc       = cv_results['test_auc'].mean()
train_pr_auc    = cv_results['test_pr_auc'].mean()
```



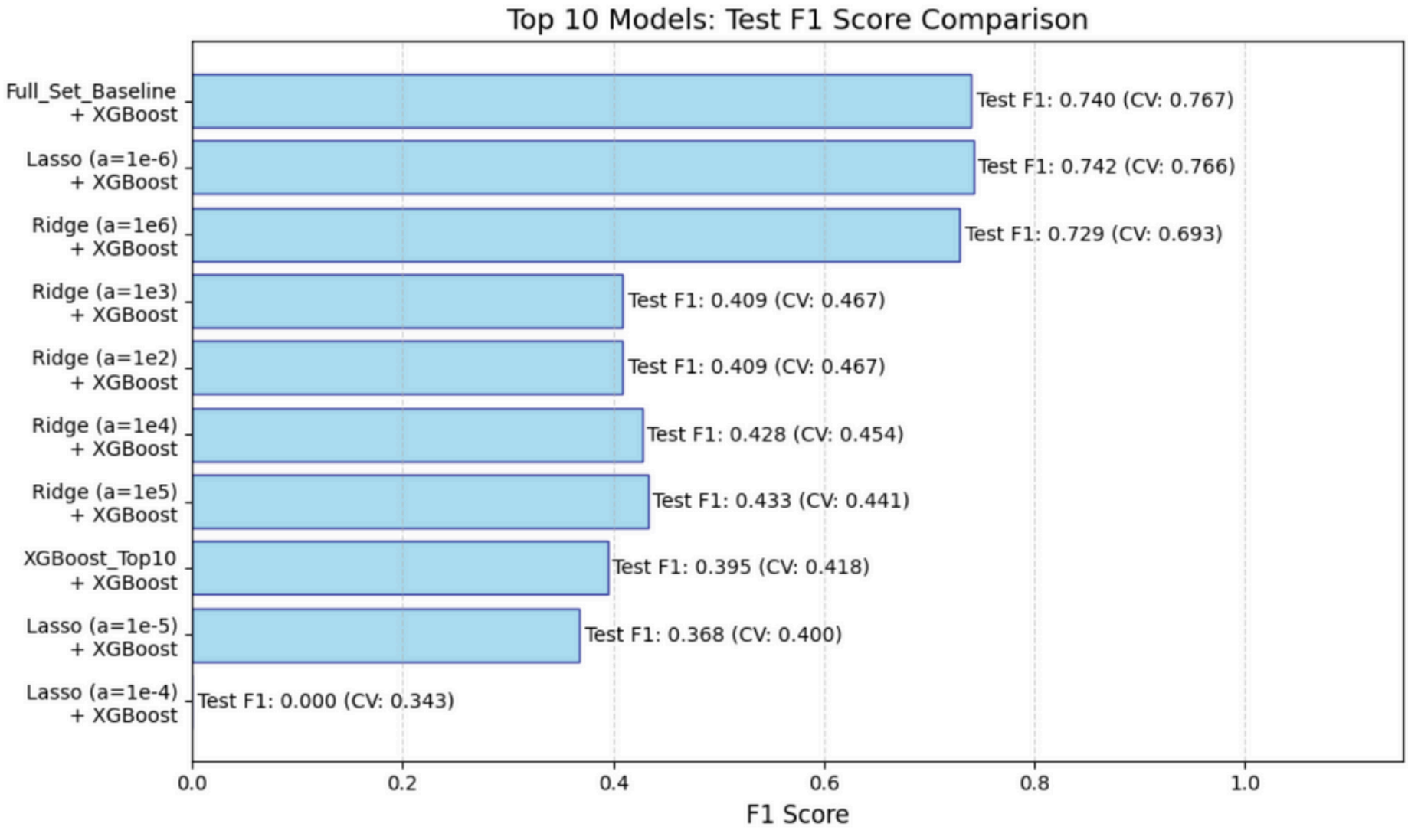
Model Training

總方法數：4x10＝40	Result02												
ML	Feature Selection	train_precision	train_recall	train_f1	train_auc	train_pr_auc	train_accuracy	test_precision	test_recall	test_f1	test_auc	test_pr_auc	test_accuracy
LogisticRegression	Full_Set_Baseline	0	0	0	0.8698	0.0138	0.9985	0	0	0	0.8683	0.0141	0.9985
	Lasso (a=1e-4)	0	0	0	0.8736	0.0132	0.9985	0	0	0	0.8729	0.0135	0.9985
	Lasso (a=1e-5)	0	0	0	0.8891	0.0167	0.9985	0	0	0	0.8884	0.0174	0.9985
	Lasso (a=1e-6)	0	0	0	0.8870	0.0168	0.9985	0	0	0	0.8861	0.017	0.9985
	Ridge (a=1e2)	0	0	0	0.8650	0.0132	0.9985	0	0	0	0.8641	0.0133	0.9985
	Ridge (a=1e3)	0	0	0	0.8650	0.0132	0.9985	0	0	0	0.8641	0.0133	0.9985
	Ridge (a=1e4)	0	0	0	0.8897	0.0179	0.9985	0	0	0	0.8886	0.0182	0.9985
	Ridge (a=1e5)	0	0	0	0.8893	0.0183	0.9985	0	0	0	0.8887	0.0187	0.9985
	Ridge (a=1e6)	0	0	0	0.8767	0.0164	0.9985	0	0	0	0.8763	0.0165	0.9985
	XGBoost_Top10	0	0	0	0.8763	0.0081	0.9985	0	0	0	0.8749	0.0081	0.9985
LDA	Full_Set_Baseline												
	Lasso (a=1e-4)	0	0	0	0.8682	0.0129	0.9985	0	0	0	0.8674	0.0133	0.9985
	Lasso (a=1e-5)	0.0438	0.1278	0.0652	0.8706	0.0247	0.9945	0.0427	0.124	0.0635	0.8684	0.0226	0.9945
	Lasso (a=1e-6)	0.0507	0.1436	0.0749	0.8666	0.0270	0.9947	0.0502	0.141	0.0741	0.8618	0.0249	0.9947
	Ridge (a=1e2)	0.0499	0.1405	0.0736	0.8683	0.0268	0.9947	0.0491	0.1373	0.0723	0.8629	0.0247	0.9947
	Ridge (a=1e3)	0.0499	0.1405	0.0736	0.8683	0.0268	0.9947	0.0491	0.1373	0.0723	0.8629	0.0247	0.9947
	Ridge (a=1e4)	0.0504	0.1421	0.0744	0.8674	0.0267	0.9947	0.0495	0.139	0.073	0.8619	0.0246	0.9947
	Ridge (a=1e5)	0.0503	0.1412	0.0741	0.8685	0.0268	0.9947	0.0495	0.138	0.0728	0.8629	0.0247	0.9947
	Ridge (a=1e6)	0.0428	0.1247	0.0638	0.8702	0.0244	0.9945	0.042	0.1217	0.0625	0.8681	0.0224	0.9945
	XGBoost_Top10	0.0453	0.1234	0.0663	0.8553	0.0232	0.9948	0.0433	0.1163	0.0631	0.8499	0.0211	0.9948
QDA	Full_Set_Baseline	0.0039	0.706	0.0077	nan	nan	0.7806	0.0062	0.7987	0.0122	0.8631	0.0195	0.8067
	Lasso (a=1e-4)	0	0	0	0.8694	0.0129	0.9985	0	0	0	0.8669	0.0134	0.9985
	Lasso (a=1e-5)	0.0102	0.7058	0.0201	0.9039	0.0241	0.8946	0.0099	0.7077	0.0196	0.9035	0.0218	0.8941
	Lasso (a=1e-6)	0.0183	0.5766	0.0354	0.9231	0.0247	0.953	0.0183	0.5755	0.0354	0.9196	0.0221	0.9531
	Ridge (a=1e2)	0.0096	0.6895	0.0189	0.8946	0.0206	0.8914	0.0091	0.6907	0.018	0.8852	0.0179	0.8875
	Ridge (a=1e3)	0.0145	0.607	0.0281	0.8945	0.0201	0.9266	0.0097	0.7103	0.0191	0.8898	0.0174	0.8907
	Ridge (a=1e4)	0.0096	0.7787	0.019	0.9157	0.0247	0.8794	0.0093	0.8568	0.0184	0.9082	0.0226	0.8633
	Ridge (a=1e5)	0.008	0.8732	0.0158	0.9075	0.0237	0.8366	0.0082	0.7612	0.0163	0.903	0.0216	0.8624
	Ridge (a=1e6)	0.0075	0.8857	0.0149	0.9092	0.0250	0.8231	0.0087	0.8393	0.0173	0.9037	0.0229	0.8573
	XGBoost_Top10	0.0295	0.2656	0.0531	0.9106	0.0209	0.9858	0.0288	0.2597	0.0519	0.9071	0.0187	0.9858
XGBoost	Full_Set_Baseline	0.8552	0.695	0.7667	0.9380	0.7426	0.9994	0.8285	0.669	0.7402	0.909	0.6729	0.9993
	Lasso (a=1e-4)	0.8412	0.2155	0.3429	0.9286	0.2481	0.9988	0	0	0	0.7242	0.0116	0.9966
	Lasso (a=1e-5)	0.6035	0.299	0.3996	0.8597	0.3043	0.9987	0.5858	0.268	0.3678	0.8222	0.2509	0.9986
	Lasso (a=1e-6)	0.8508	0.6973	0.7664	0.9363	0.7349	0.9994	0.8181	0.6793	0.7422	0.913	0.686	0.9993
	Ridge (a=1e2)	0.675	0.3567	0.4666	0.8987	0.3944	0.9988	0.6167	0.3058	0.4088	0.8276	0.3317	0.9987
	Ridge (a=1e3)	0.675	0.3567	0.4666	0.8987	0.3944	0.9988	0.6167	0.3058	0.4088	0.8276	0.3317	0.9987
	Ridge (a=1e4)	0.6425	0.3509	0.4539	0.8820	0.3687	0.9987	0.5801	0.3385	0.4275	0.8579	0.346	0.9986
	Ridge (a=1e5)	0.6281	0.34	0.4406	0.8914	0.3414	0.9987	0.6627	0.3212	0.4327	0.9056	0.3809	0.9987
	Ridge (a=1e6)	0.7962	0.6134	0.6928	0.8963	0.6320	0.9992	0.8326	0.648	0.7288	0.9067	0.6805	0.9993
	XGBoost_Top10	0.6104	0.3184	0.4183	0.8704	0.3204	0.9987	0.6342	0.2865	0.3947	0.855	0.322	0.9987

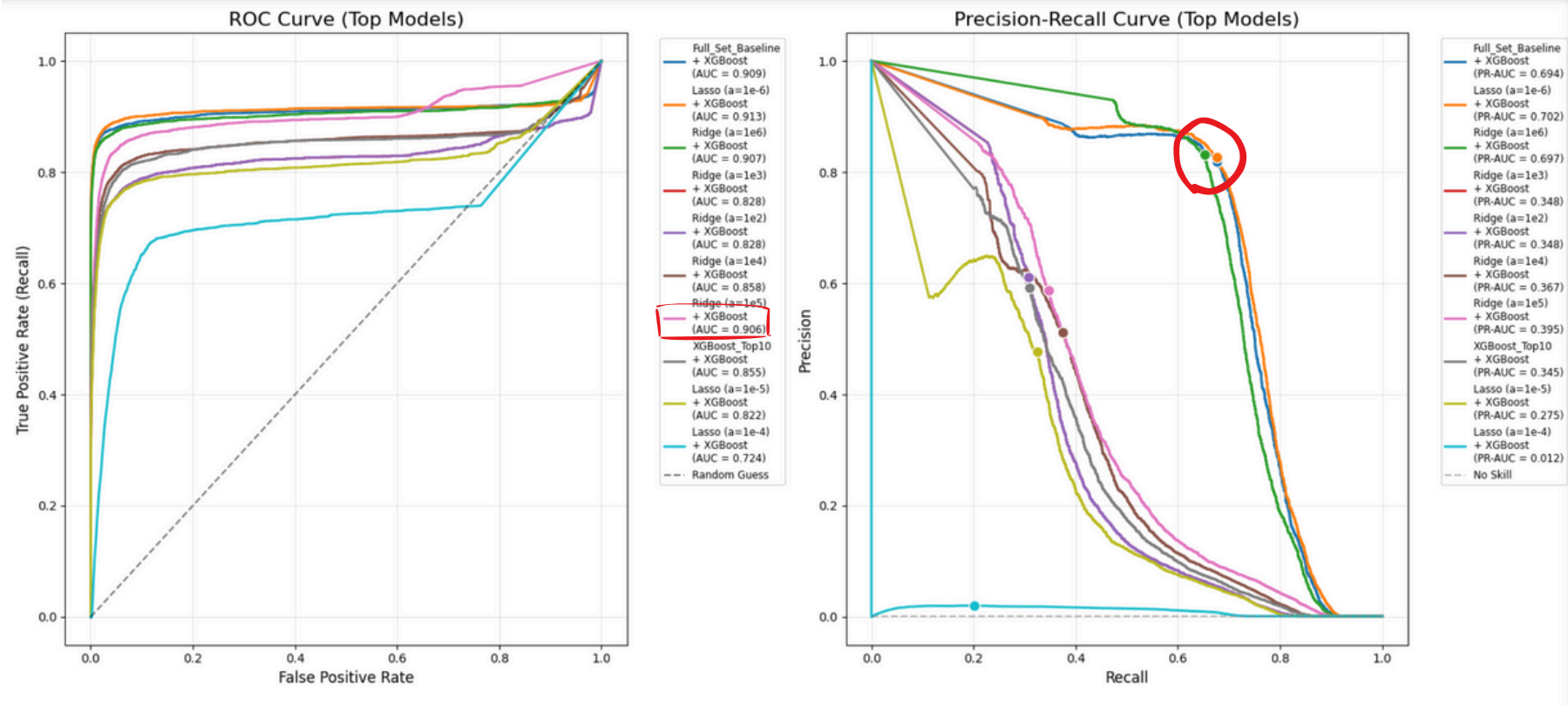


Model Evaluation

Model Training & Results

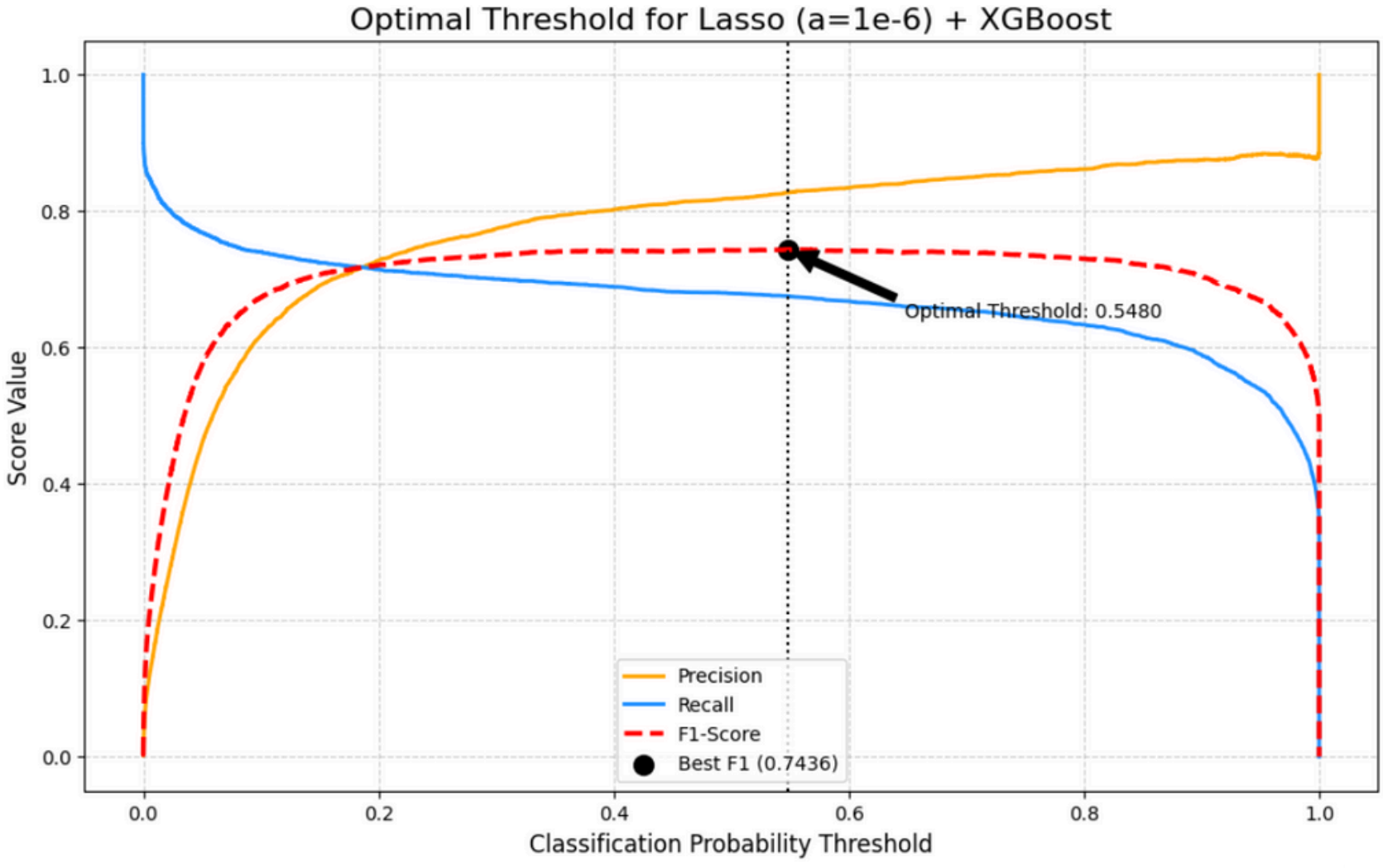


Model Evaluation



Best Model

Best Model => Lasso (a=1e-6) + XGBoost



Conclusion

Feature selection improved performance

- Lasso (α = 1e–6) removed weak features and boosted generalization.

Classical models underperformed

- Logistic, LDA, QDA had very low F1 → task is highly non-linear.

XGBoost was the best overall

- Highest F1, ROC-AUC, and PR-AUC across all feature sets.

Best pipeline: Lasso (1e–6) + XGBoost

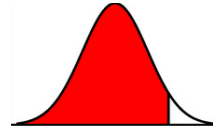
- Test F1 = 0.7436 (CV ≈ 0.767) → strong and stable model.

Strong model evaluation

- ROC-AUC ≈ 0.913 and solid PR-AUC on imbalanced data.

Threshold tuning improved results

- Optimal threshold 0.548 boosted F1 and precision–recall balance.



Next-Gen Credit Risk and Fraud Detection with Explainable AI

Cheng-Min Tsai,
Nguyen-Thu Thuy,
Huei-Wen Teng

NYCU
MY SITE